Agentforce and RAG

Best practices for better agents

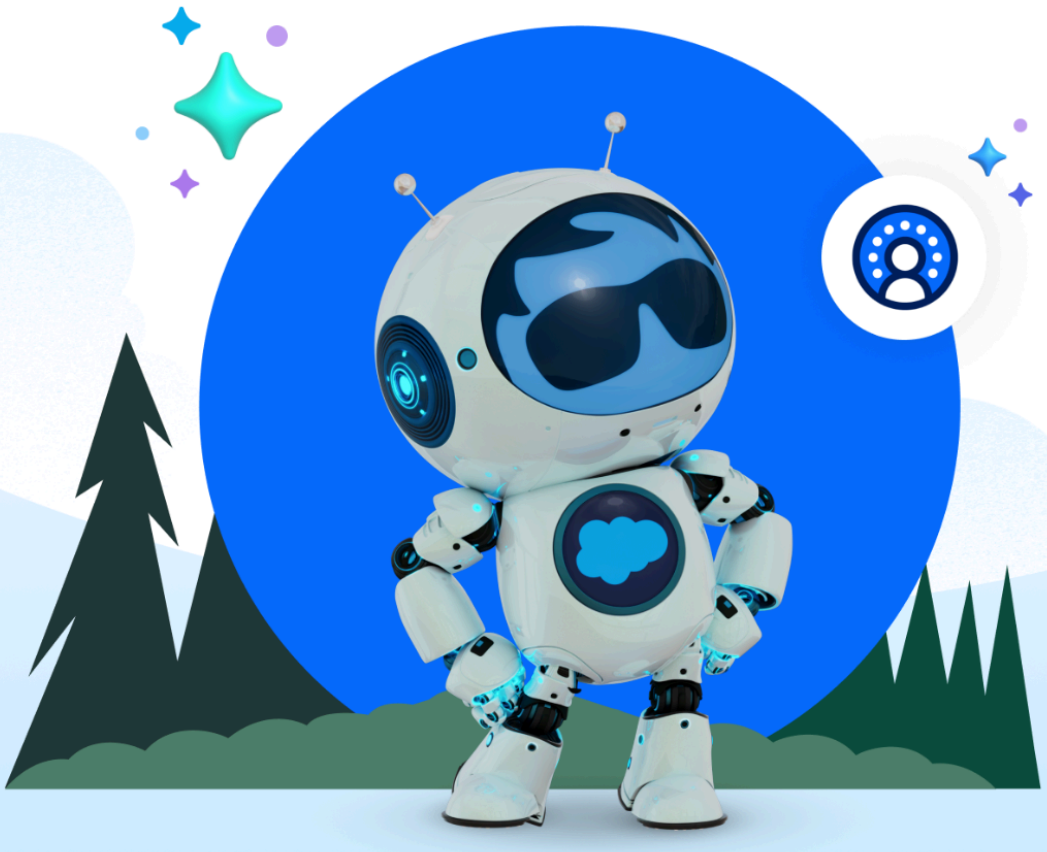


Table of Contents

1. Introduction
    1.1 Purpose of this document
    1.2 Overview of RAG on the Salesforce Platform
    1.3 Prerequisites to reading this document
2. Curating content for RAG
    2.1 What content types and platforms are supported by search indexes?
        Search Indexes for Agentforce Data Libraries (ADLs)
        Manual Search Indexes in Data Cloud
    2.2 Shaping content for optimal RAG results
    2.3 Multilingual support
3. Designing RAG-powered agents
    3.1 What happens under the hood of data libraries, what can be (re)configured?
        File-Based Search Indexes
        Knowledge Article-Based Search Indexes
    3.2 Optimal agent design for RAG: ADL and manual config. What if there are multiple data sources?
        Advantages of ADL
        Limitations of ADL
        Manual Setup
        Design Choices for RAG-Based Solution Architectures
            Approach 1: One prompt template (and corresponding agent action) with all retrievers.
            Approach 2: A separate prompt template for each retriever
            Combining Approaches 1 and 2
        Topic and action descriptions for RAG
4. Hybrid Search
    4.1 What is hybrid search?
    4.2 When to use hybrid search and when not to?
    4.3 What are the implications of hybrid search?
    4.4. How can hybrid search be optimized?
5. Optimize Search Indexes: Field Selection, Chunking, and Vectorization
    5.1 What kinds of fields are important to RAG solutions?
    5.2. How many index fields to select, and which ones? What happens to them?
    5.3 How to optimize the chunking strategy?
        Field prepending
        Size
            Optimize for retrieval
            Optimize for augmentation
    5.4 What is chunk enrichment and when should it be used?
    5.5 Which embedding models should be used?
        E5-Large V2 (open source)
        Multilingual E5-Large (open source)
        (Roadmap) Open AI Ada 002

5.5 RAG on documents belonging to one record
6. Configuration of the retriever
6.1 How can prompts be grounded with data outside of the retriever/index?
6.2 How can I apply filters to the results? And what are dynamic prefilters?
Pre-retrieval and Post-retrieval Filters
Dynamic Prefilters
6.3 What is advanced retrieval mode and when should I use it?
7. RAG and Prompt Engineering
7.1: What does a good RAG prompt template look like?
Example of a Basic Prompt Template
Answer Question with Knowledge Prompt Template
8. Citations (roadmap)
8.1 What are the different options for setting up citations?
9. Pro-code RAG without retrievers
9.1 What are the limitations of (no-code) retrievers?
9.2 How to perform RAG without retrievers?
10. Selecting the right LLM for RAG
10.1 What LLM is best for RAG? What LLM parameters matter?
Context Window Size
Reasoning Capabilities
11. RAG in Flow
11.1 How can I use retrievers in flow? What does flow mean for RAG?
12. Debugging and Troubleshooting RAG-Powered agents
12.1 Troubleshooting Solution Layers
Step 1.Agent Layer: Are the agent topic and agent action being called?
Step 2. Agent Layer (ADL only): Is the right retriever passed to the action?
Step 3. Search Index Layer: Does the search index contain vectors?
Step 4. Retriever Layer: Is the retriever augmenting the prompt with content?
12.2 Interpreting RAG Evaluation Metrics roadmap
Metrics
Common Patterns In Quality Metrics
High Faithfulness, Low Context Relevance
Low Faithfulness, High-Context Relevance
High Faithfulness and High-Context Relevance, Low-Answer Relevance
13. General things worth knowing
13.1 Resolution without generating an LLM response
13.2 Non-RAG use cases with retrievers: document identification
13.3 Non-RAG use case with search indexes: classification
Appendix
Forward-looking statements
Summary of provided best practices
Acknowledgements

1. Introduction

1.1 Purpose of this document

The purpose of this document is to provide best practices for powering Agentforce with unstructured data (files) and long free-text fields on structured objects. The Salesforce Platform offers a collection of integrated tools to set up and implement retrieval augmented generation (RAG)-powered solutions. RAG ensures that agent responses are more accurate, up to date, and relevant with respect to a company's enterprise knowledge[1]. Some of these tools are straightforward and out of the box. Some allow granular configuration for additional control over the solution's behavior. In all cases, however, implementers are required to make choices about solution architecture and design. The purpose of this document is to guide the Salesforce admin or developer in making these design choices in the best way given their requirements.

The content in this document is inspired by real-life scenarios and implementations. It describes technology that is generally available (GA) or open beta at the time of writing (see date of last update at the beginning of the document). Occasionally, roadmap items will be discussed when they are close to being released. In these cases, the content is marked as a roadmap item, if possible with an indication of the release timeline, subject to the forward-looking statements listed in the appendix.

This document in no way replaces the documentation on help.salesforce.com and information on Trailhead. Both sources describe the capabilities of RAG on the Salesforce Platform. Instead, this document *complements* that documentation and offers guidance on how to best use these capabilities, resulting in high-performing agents.

For completeness, this introduction contains a brief overview of RAG on the Salesforce Platform. However, this introduction is not complete as a beginner's guide. Therefore, this chapter ends with a set of prerequisites needed for a sufficient understanding of the rest of this document.

1.2 Overview of RAG on the Salesforce Platform

Retrieval augmented generation (RAG) in Data Cloud is a framework for grounding prompts for large language models (LLM). By augmenting the prompt with accurate, current, and pertinent information, RAG improves the relevance and value of LLM responses for users. There are different ways of bringing data to a prompt (e.g., merge fields, data graph). When we speak of RAG in this document, we explicitly refer to augmenting a prompt with long-form textual content that is retrieved based on semantic similarity to a query.

---

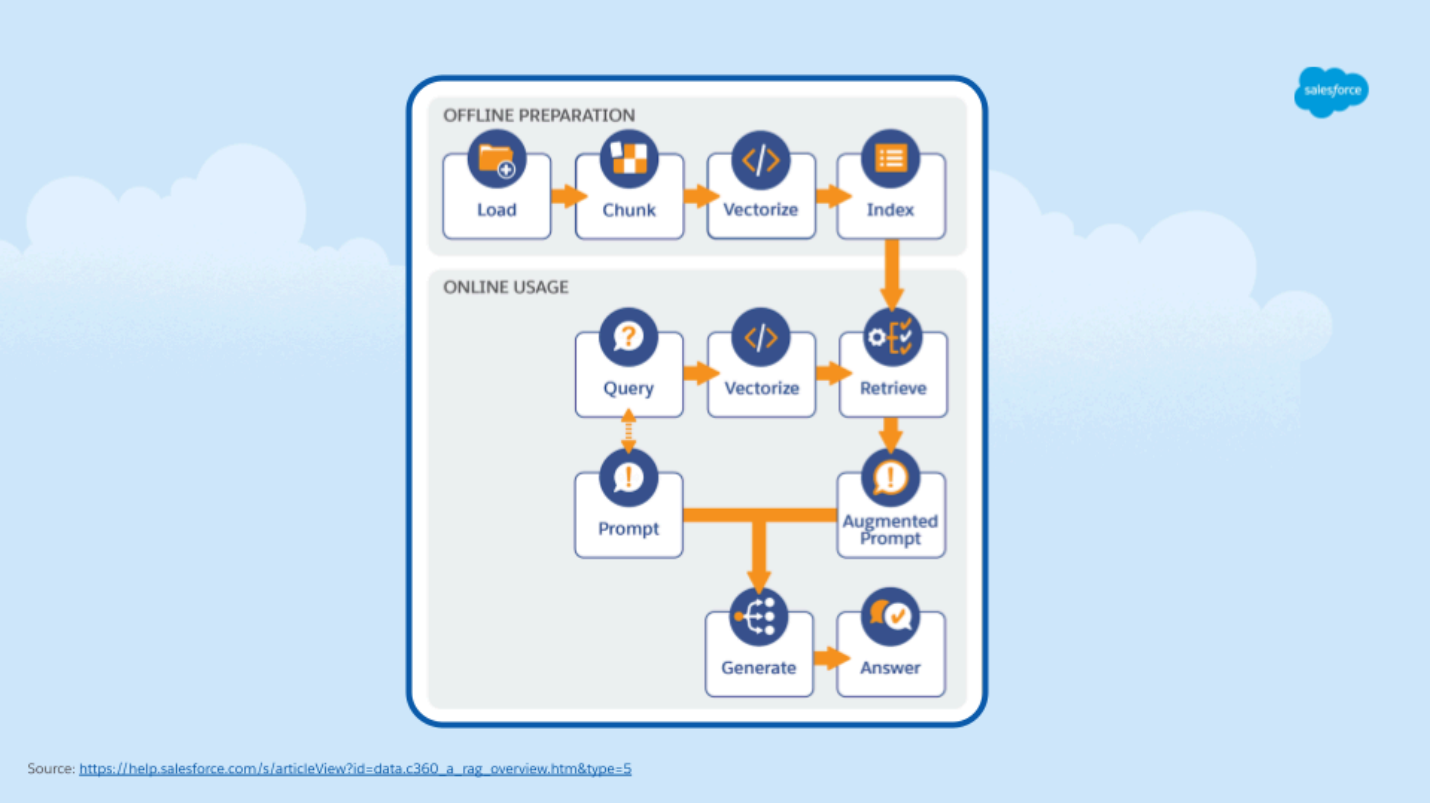
[1] When we say knowledge we don't only mean knowledge articles, but the wider data corpus of all enterprise knowledge spread over files, emails, articles, documents, call notes or transcripts, object descriptions, long/free text fields in structured tables, etc.

When you submit an LLM prompt, RAG in Data Cloud:

- Retrieves relevant information from a knowledge store (called a search index) containing structured and unstructured content
- Augments the prompt by combining this information with the original prompt
- Generates a prompt response

Many LLMs were trained generally across the internet on static and publicly available content. RAG adds information to a prompt that's accurate, up to date, and not available as part of the LLM's trained data (for example, a company's private data). It supplements the LLM's capabilities with relevant information retrieved from a knowledge store. With RAG, prompt template users provide proprietary data to the LLM without needing to retrain and fine-tuning the model. The result is generated LLM responses that are more pertinent to the user's context and use case.

It's helpful to think of RAG in two main parts: offline preparation and online usage.



The notion of a *query* is critical here. The query is a part of the prompt that contains the search string. This search string reflects the user's intent, which the RAG process uses to retrieve relevant data. This retrieval process is based on *semantic similarity,* so based on the meaning of the query and the retrieved data instead of the exact words. Hybrid search extends semantic search with keyword similarity, which can improve LLM response quality. RAG therefore differs from just looking up data using an identifier or even a single keyword. It always involves identifying longer, free-form textual data based on semantic similarity with a search string that contains more than just a single keyword.

Quick Start for Offline Preparation

The fastest way to set up your RAG solution is to add an Agentforce Data Library (ADL), either in Agent Builder or Setup. Creating an ADL automatically sets up all of the components needed for a working, RAG-powered solution: data streams, objects and mapping, vector data store, search index, retriever, prompt template, and the agent action. Salesforce uses default settings for these components. Alternatively, you can use the created components as a basis for further configuration and refinement, such as using the default retrievers in custom templates, or creating custom retrievers for the created search index.

Advanced Setup for Offline Preparation

To implement RAG in Data Cloud via manual configuration (not using ADL), start by connecting the structured and unstructured data from which RAG retrieves relevant information for LLM prompt grounding. Data Cloud uses search indexes to manage structured and unstructured content in search-optimized ways. Content in supported file types can be ingested from a variety of sources. Some examples of unstructured content used with RAG include service replies, cases, RFP responses, knowledge articles, FAQ, emails, and meeting notes.

Offline preparation involves these steps in Data Cloud:

1. Connect your unstructured data (DSO → DLO/UDLO → DMO/UDMO).
2. Create a search index configuration to chunk and vectorize the content.
   Chunking breaks the text into smaller units that reflect passages of the original content, such as sentences or paragraphs. Vectorization converts chunks into numeric representations of the text that capture semantic similarities.
3. Store and manage the search index in Data Cloud.

To learn more, see Search for AI, Automation, and Analytics.

Retrievers

Retrievers serve as the bridge between search indexes and prompt templates. When you create a search index, Data Cloud automatically creates a default retriever for it that is visible in its Einstein Studio. To support a variety of use cases, you can create custom retrievers in Einstein Studio. Custom retrievers further refine the search criteria and retrieve the most context-relevant information used to augment prompts, for example, by adding filters or including additional return fields. To learn more, see Manage Retrievers.

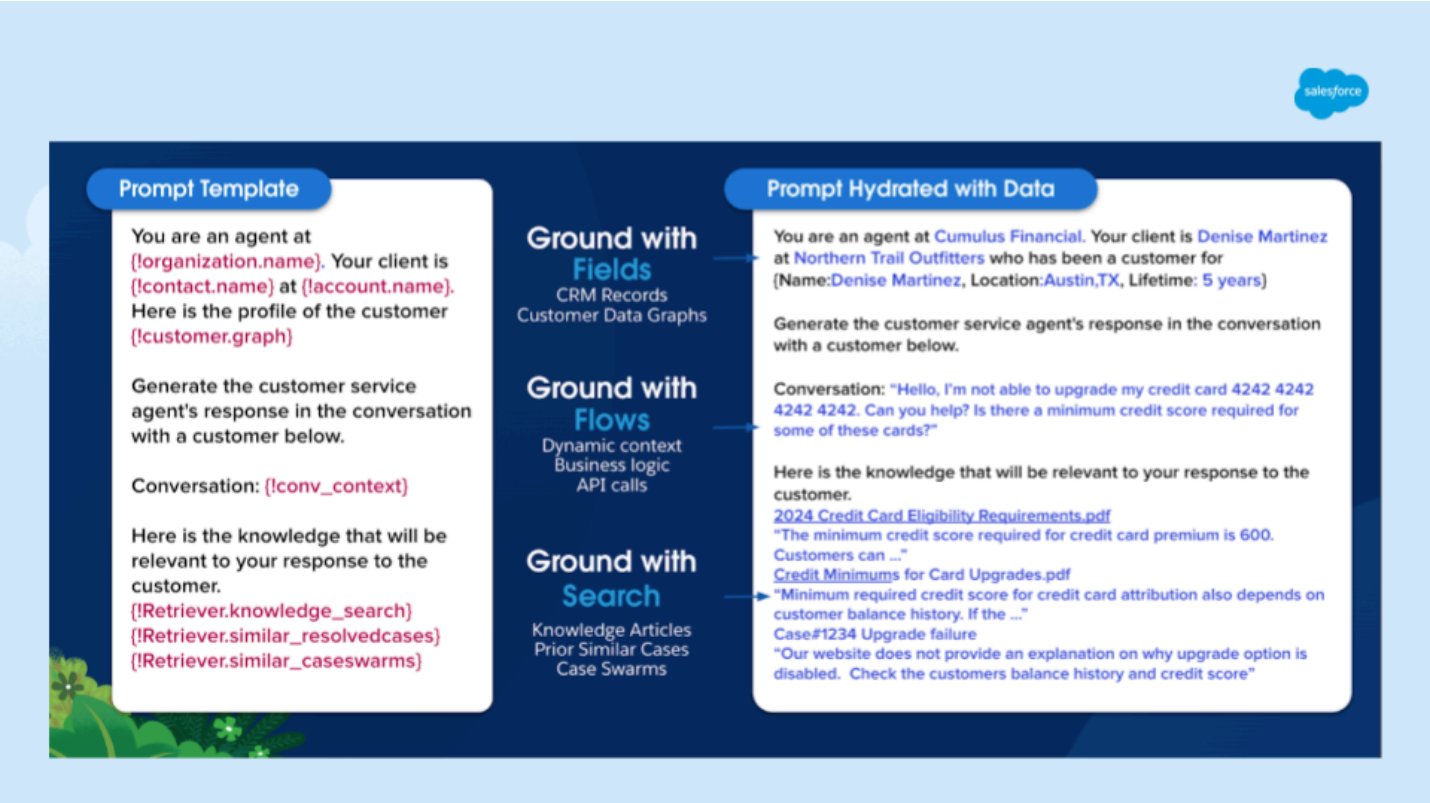
Online Usage

The final piece of the RAG implementation is to add a call to a retriever in a prompt template. For a given prompt template, the prompt designer can customize retriever query and results settings to populate the prompt with the most relevant information. To learn more, see Ground with Retrieval Augmented Generation (RAG) in Data Cloud.

Each time a prompt template with a retriever is run, this sequence occurs (see also diagram above):

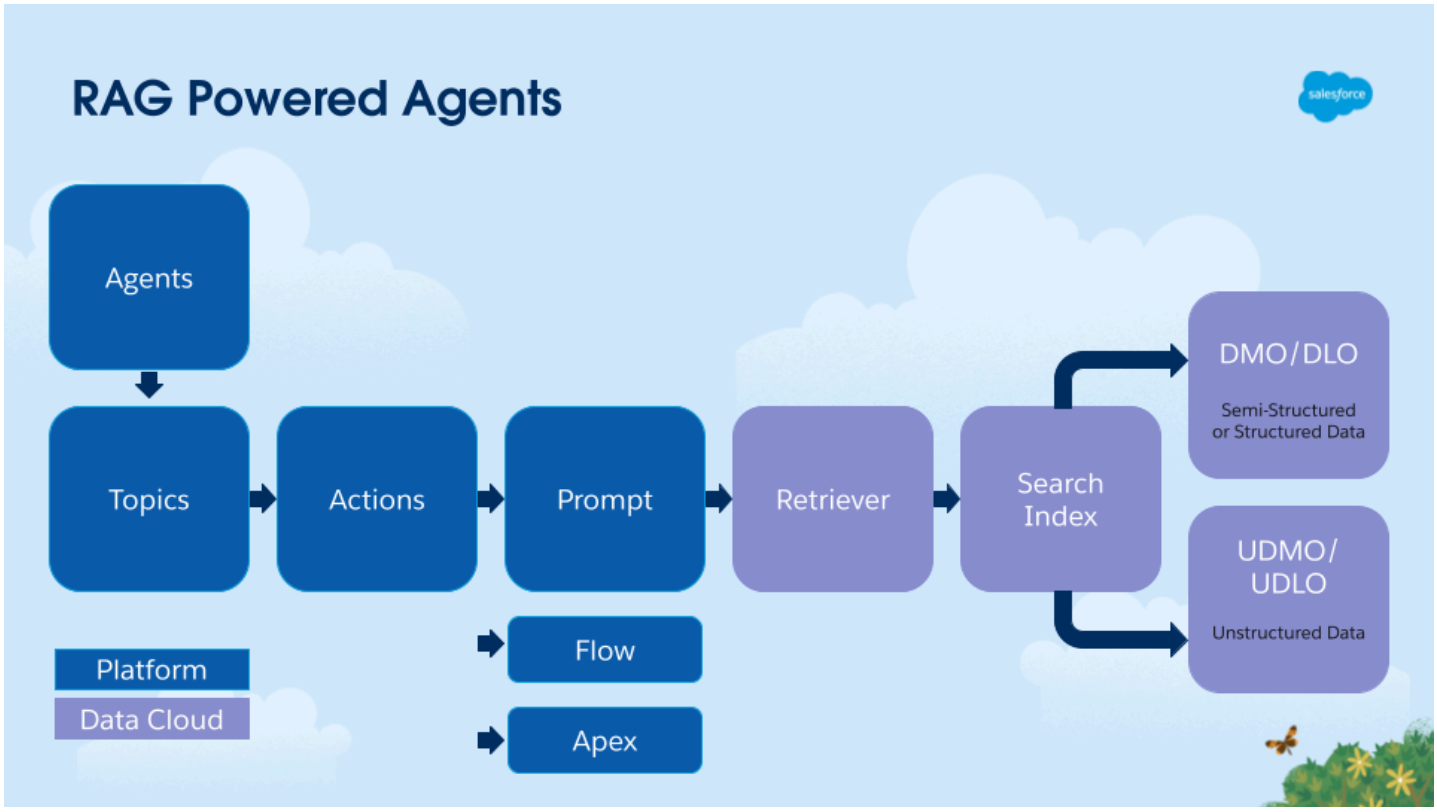
1. The retriever is invoked with a dynamic query initiated from the prompt template.

2. The query is vectorized (converted to numeric representations). Vectorization enables search to find semantic matches in the search index (which is already vectorized).
3. The query retrieves the relevant content from the indexed data in the search index.
4. The original prompt is populated with the information retrieved from the search index.
5. The prompt is submitted to the LLM, which generates and returns the prompt response.



RAG-Powered Agents

So far we have described the offline preparation of the search index and its online usage within a Prompt template. To bring RAG to Agentforce, an agent action is needed to call this prompt template. This figure shows the entire run-time flow for RAG-powered Agents.



(note: section 1.2 draws upon the introductory content from help.salesforce.com)

1.3 Prerequisites to reading this document

Review the following content to better understand of the remainder of this document:

- Complete Trailhead Module: Retrieval Augmented Generation: Quicklook
- Complete Trailhead Module: Unstructured Data in Data Cloud
- Watch Demo video: From Unstructured Data to Agentforce
- Read Documentation: Add a Data Library
- Read Documentation: Search for AI, Automation, and Analytics.
- Read Documentation: Manage Retrievers
- Read Documentation: Ground with Retrieval Augmented Generation (RAG) in Data Cloud.

2. Curating content for RAG

Structured content. Structured content refers to objects and data tables that conform to a meaningful structure, with values and relationships, including categorical (picklists, identifiers), numerical, and referential fields. Some structured content includes long-form text fields, such as a description, conversation, or article. Only these, where the text contains at least one full sentence representing a semantically meaningful factoid, can be used for RAG. Categorical and numerical fields related to the indexable text, such as sensitivity level, view count, product type etc. can be used for enriching the index, as described in Section 5.1.

Unstructured content. Files of long-form text (documents, articles, emails, notes, and so on) are referred to as unstructured content. They can even be audio and video files, whose textual content is transcribed and then used for the RAG process. The audio and video files remain at their respective locations (zero-copy file stores). They are submitted to a transcription service that converts the speech into text. That transcription is chunked and vectorized, so that only the chunks and vectors are stored in Data Cloud.

Not all files are by definition unstructured. If the file contains an inherent structure with fields and values, such as JSON, CSV, or XML formats, the file should be loaded as structured data first, and only the long-form texts (if any) can be used for chunking and vectorization. The other fields can be used for index and retrieval enrichment as described in Section 5.1.

Curation. To improve LLM prompts and responses, shape the source content in ways that optimize information retrieval. Certain product features address problems encountered in content that is not well formed, such as enriched indexing (Section 5.4) and field prepending (Section 5.1). However, applying content curation practices to source content can enhance retrieval for improved results.

2.1 What content types and platforms are supported by search indexes?

Search Indexes for Agentforce Data Libraries (ADLs)

- Salesforce knowledge articles
- Uploaded files in the following formats
    - PDF (max 100mb)
    - HTML (max 4mb)
    - txt (max 4mb)

Manual Search Indexes in Data Cloud

Additional content sources are supported for search indexes created manually in Data Cloud:

- Text fields containing free text on any DMO (e.g., originating from sObjects using the CRM connector, or ingested into Data Cloud from other systems)
- Related files from Salesforce records
- Salesforce knowledge articles (using the pattern of ingesting them as a DMO in Data Cloud)
- Supported file formats:
    - PDF (max 100mb)
    - HTML (max 4 mb)
    - txt (max 4 mb)
    - Audio (mp3, wav, flac, mpga, m4a, ogg)
    - Video (mp4, mpeg, webm)
    -

- Supported storage platforms: Files are supported from the following platforms (note that the files are *not* copied over to Data Cloud[2]):
    - AWS S3
    - Google Cloud Storage
    - Azure Blob Store
    - (roadmap) Via MuleSoft direct connectors:
        - Sharepoint
        - Google Drive (including Google docs)
        - Confluence
        - Sitemap

2.2 Shaping content for optimal RAG results

Document best practices (including knowledge).
The following recommendations apply to documents and long-form text stored in long text fields, such as knowledge articles or other objects that contain long-form text. These recommendations can be applied whether writing or generating articles. In the latter case, simply provide these recommendations as instructions to the generating prompt.

1. **High level of detail, thorough explanations.** Generative AI works best with thorough, complete information that it can synthesize and report back to the user at differing levels of detail, tailored to a specific audience (as defined in the prompt instructions). For example, customers chatting with an agent often want a short, direct answer. A service rep diagnosing a complex issue, however, often prefers more comprehensive technical information. In general, for better results, favor being too thorough and detailed in your knowledge content over being too brief. Similarly, providing detailed contextual information helps with retrieving the right content. For example, explaining common synonyms and abbreviations in articles helps the large language model (LLM) better understand how the different concepts in your knowledge content relate to each other.
2. **Provide real-world examples.** In knowledge articles, include examples a typical user would face for the given topic and describe them in detail. Providing these scenarios in a conversational way helps the retrieval process identify the right content better in response to user queries.
3. **Structure the articles well.** Ensure that sentences are logically related to each other. Organize content into paragraphs that make sense and cohere rationally. For HTML-formatted content, use heading tags (H1 to H6) to signal how content is hierarchically related. The chunking process uses heading tags as chunk delimiters.
4. **Spread content over fields in Salesforce knowledge articles.** When chunking and vectorizing Salesforce knowledge articles, the search index is built against a structured DMO. Take advantage of this structure by spreading long-text content across multiple fields (for example, Question, Description, Resolution, and Exceptions). Annotate the knowledge article with metadata that can be used for filtering and prepending, as explained in Section 5.1.

[2] This means the physical files will not reside on Data Cloud. The raw textual contents from the chunking process reside on Data Cloud. DMOs with chunks also reside on Data Cloud.

5. **Annotate media.** Video files and images included in documents are ignored by the chunking and vectorization process. However, their associated descriptions do get chunked and vectorized. Take advantage of this by using alt text and richly describing the contents of the media so their meaning and value get included in the search index.
6. **Keep content focused and structured to align with common user questions.** RAG is often used to answer user questions, such as in FAQ agents. The system then needs to establish the semantic relationship between content and user query. Linking questions and answers explicitly in the document helps to amplify semantic connections, because question and answer are then vectorized together. For other documentation approaches, such as a typical user manual that only contains content, the semantic match between question and content is less clear. Similarly, segment content so that articles are focused on isolated topics. Don't combine multiple unrelated (or only slightly related) topics into a single article.
7. **Use titles, headings, and subheadings.** Titles, headings and subheadings help provide valuable context to the content. Even better is to store this information in additional fields for use in prepending (see Section 5.1). Although this context can be generated by the enriched chunking functionality (see Section 5.4), it's better to include it explicitly in the document than to rely on LLMs to add this context.
8. **Perform knowledge audits and apply governance.** Grounding on incorrect, outdated information results in AI-generated prompt responses that are inaccurate and out of date. It's best to conduct regular audits of your knowledge base and quickly remedy any issues encountered. Use versioning for knowledge content and be sure to manage versions to govern the search index content.
9. **Convert complex tables to JSON or HTML, split long tables.** Many documents include embedded tables that occur throughout the text. When tabular data is not explicitly formatted (such as in a PDF document), it can help to format the data first as JSON. Use an LLM, which understands tables, and reformat them using an explicit structure, such as JSON or HTML. Doing so helps during the LLM that is used in RAG to generate a better response. For long tables, consider splitting them up into smaller tables and repeating the header, which helps with chunk delineation.

2.3 Multilingual support

The search index supports content in many different languages, thanks to the multilingual embedding models as described in detail in Section 5.5. It supports dozens of different languages and preserves semantic similarity across languages. That means a query can be in one language, retrieving semantically relevant results from content that is written in another language. The results may vary depending on the presence of that language in the training data set of the embedding model. More information on this is in Section 5.5.

Languages can also be used as prefilters (see section 5.1). This is helpful when the content should not be used across languages and responses should be generated using content from the same language as the query.

Note that language support in Agentforce is a broader topic and that the support of languages differs per feature.

- **Unstructured data and RAG** (embedding and indexing): supported languages for chunking and vectorization for semantic and lexical similarity search.
    - Dozens of different languages are supported. See for more details Section 5.5.
- **Prompt template input**: grounding languages supported by the LLM that generates the RAG response through prompt templates.
    - The LLM that generates a response to the RAG prompt defines the supported languages. The default models available in Prompt Builder support the same languages as the multilingual embedding models.
- **Prompt template output**: output languages supported by the prompt runtime to generate responses.
    - See language support here.
- **Trust Layer:** trust components that are language specific, such as toxicity detection or data masking.
    - See language support here.
- **Agentforce**: languages supported by the Agentforce platform (conversational experience and reasoning engine).
    - See language support here.

3. Designing RAG-powered agents

3.1 What happens under the hood of data libraries, what can be (re)configured?

The fastest way to set up a RAG-powered agent is to configure an Agentforce Data Library (ADL). For ADL (either in Setup or Agent Builder), the user uploads files to the ADL or selects knowledge articles. It isn't possible to add other content to the ADL[3]. For example, to retrieve content from Salesforce objects other than knowledge articles, the RAG pipeline needs to be configured manually.

Files are manually uploaded to ADL and stored on a file storage managed by Salesforce. The file contents are used for RAG. Files can't be downloaded from that storage, nor is there a clickable link to bring the user back to that file. It resides there until deleted by a user.

After configuring the ADL, all assets for the RAG pipeline are automatically created using the default settings described below.

[3] Open web search is coming to ADL in early 2025, but that feature will not ingest data and build search indexes, so it's left out of the scope of this document for now.

File-Based Search Indexes

- **Data Cloud Objects**
  - The unstructured data model object for files is named `FileUDMO__dlm`. This object does not contain the actual file, but only the metadata (for example, file type, file location).
- **Search Index**
  - There is one search index in the org named `FileUDMO_SI` for all ADLs. Files from different ADLs are organized in the same search index. The corresponding index DMO object (containing the vector), named `FileUDMO_SI_index__dlm`, contains a reference to the ADL via a field named `GroundingSourceId__c`. This grounding source ID represents the ADL to which that the vector belongs. A special DMO named `AiGroundingFileRefCustom__dlm` contains the mapping between uploaded files and their ADL. It's possible to create a search index manually for `FileUDMO__dlm` and use different settings. The following settings are used for the default search index for ADL:
    - Hybrid search
    - No enriched chunking (roadmap opt-in to enriched chunking will be available in ADL late February 2025)
    - 512 tokens per chunk
    - E5 Large Multilingual embedding model
- **Retriever**
  - `Default FileUDMO_SI`: As with every search index, there is a single default retriever that retrieves from all indexed content. It's created with the following default settings:
    - Returns 10 results
    - For each result, it returns the following fields: Key Qualifier Record ID, Data Source, Chunk, Record ID, Data Source Object, Chunk Sequence Number, Internal Organization, and Source Record ID. (See this help.salesforce.com page for details about the contents of these fields.)
    - Advanced retrieval mode (see Section 6.3) is switched off.
  - For every ADL, there is a separate retriever. This retriever is prefiltered using the grounding source ID field to retrieve content only from the corresponding ADL. It returns the following fields: Chunk, SourceRecordId, DataSource, and DataSourceObject. All other settings match the settings of the default retriever.
  - Manually created retrievers: It is possible to create retrievers manually for the `FileUDMO_SI` search index. These can then be used for grounding custom prompt templates.

Knowledge Article-Based Search Indexes

- **DLO/DMO.** The search index is based on a DMO named `KnowledgeArticleVersion__dlm`. This DMO is created automatically together with its data stream and mapping, if it doesn't already exist.
- **Search Index**
  - Names of a search index for knowledge articles is prefixed with `KA_` , followed by the name given by the user. For example `KA_Agentforce_Default_Library` or `KA_Published_Articles`. One search index may contain vectors for different ADL, if and only if these have been created using the same identifying fields. The identifying fields are

selected by the user and then prepended to every chunk, meaning they are actually added to the chunk text. For more information on field prepending, see Section 5.3. The content fields are the fields that are actually chunked and vectorized (after the identifying fields have been prepended to them). In the following example, ADL 1 and ADL 2 are both added to the same search index, because they have the same identifying fields. For ADL 3, a separate search index is created.

    - ADL 1:
        - Identifying Fields: Title, Summary
        - Content Fields: Detail, Answer
    - ADL 2:
        - Identifying Fields: Title, Summary
        - Content Fields: Detail, Content, Question
    - ADL 3:
        - Identifying Fields: Title, Category
        - Content Fields: Detail, Answer

- **Retriever.** The ADL knowledge settings are translated into a query template for the retriever. A retriever for an ADL with knowledge has no prefilter; the retrieval settings are maintained in the query template. It ensures the selected fields are returned and the content is coming from the correct knowledge article fields, following the filters defined on the ADL. The query template for a retriever can be seen in Einstein Studio. The following example illustrates how it retrieves chunks that originate from the fields `'FAQ_Internal_Comments_c__c'` and `'AssignmentNote__c'`.

```
Unset
SELECT v.Hybrid_score__c AS Score, c.Chunk__c AS Chunk, c.SourceRecordId__c
AS SourceRecordId, c.DataSource__c AS DataSource, c.DataSourceObject__c AS
DataSourceObject FROM
hybrid_search(TABLE(KA_Agentforce_Default_Library_index__dlm),
'{!$_SEARCH_STRING}', 'Language__c=''{!$_LANGUAGE}'' AND
KnowledgePublicationStatus__c=''Online'' AND DataSource__c in
(''FAQ_Internal_Comments_c__c'',''AssignmentNote__c'')', 30) v INNER JOIN
KA_Agentforce_Default_Library_chunk__dlm c on c.RecordId__c=v.RecordId__c
INNER JOIN ssot__KnowledgeArticleVersion__dlm kav on
c.SourceRecordId__c=kav.ssot__Id__c ORDER BY Score DESC LIMIT 10
```

Retrievers have two paths to Data Cloud objects: files and knowledge articles. For the remainder of the RAG solution stack, these paths converge in the prompt template and agent action.

- **Ensemble Retriever (ROADMAP; for now ADL is single source only)**
    - If an ADL has both knowledge articles and files, then the ADL has two retrievers, one for each path. When augmenting prompts with context, these need to be combined. One option is to ground the prompt with two retrievers, but this is suboptimal because the

output always adds both knowledge article and file results. However, some queries are best answered with just the knowledge article context or the file context, making the other irrelevant. Diffusing the prompt with irrelevant data reduces the response quality, consumes unnecessary Einstein Requests, and increases latency. Flooding the prompt with extraneous input also risks exceeding an LLM's context window, requiring a reduction of the result set sizes that the retrievers return. To address these problems, when an ADL is created, an ensemble retriever is created by default. An ensemble retriever bundles the two retrievers into a single retriever that merges the two result sets into a single set of results, and then dynamically reranks these results based on similarity to the query. Benefits include:

    - Only one retriever is used to ground the prompt template.
    - The most relevant query results are surfaced at the top of this ranking, irrespective of the data source from which they come.
    - No irrelevant results are added to the prompt.
    - The prompt consumes fewer Einstein Requests, which reduces latency and cost.

- **Prompt template**
    - ADL creation automatically instantiates a prompt template named `Answer Questions with Knowledge`. This prompt template contains pre-written RAG instructions and takes, as input, a free-text question that it receives from the Agentforce reasoning engine. A dynamic retriever named `{!$EinsteinSearch:sfdc_ai__DynamicRetriever.results}` augments the prompt with context at run time. The Agentforce reasoning engine selects the correct retriever at runtime, based on the selected ADL of that Agent. Different agents can use this prompt template, and each agenthas its own, discrete ADL (An agent can only have one ADL). Dynamic retrievers add run-time flexibility and versatility to RAG solutions. We recommend using the default retriever provided in this prompt template instead of overriding it with a manually created retriever.
- **Agent Action**
    - ADL creation automatically instantiates a prompt template agent action named `Answer Questions with Knowledge`. It's associated with the prompt template of the same name.

3.2 Optimal agent design for RAG: ADL and manual config. What if there are multiple data sources?

ADL is recommended when an agent needs to ground its responses to questions in content from Files on a Salesforce managed File store and Salesforce knowledge articles. For other data sources, manual setup is required.

Advantages of ADL

- Streamlined, quick start approach. Automatically instantiates all the necessary components with default settings.
- Aggregates storage. No separate storage for files.

- (roadmap) Dynamic reranking. Ensemble retrievers dynamically rerank results coming from different data sources (knowledge articles and files), which improves RAG quality and reduces costs and latency.

Limitations of ADL

- Supports only files and knowledge articles. Use manual setup for other source content.
- Reduced customization options. We recommend keeping default settings across the RAG solution components. We don't advise breaking the chain by, for example, creating custom retrievers or overriding the dynamic ensemble retriever in the standard prompt template with manually created retrievers.

Manual Setup

RAG assets should be manually configured in the following scenarios:

- Different data sources are needed, such as files already stored in external blob stores or other document shares, and long-text fields on objects other than knowledge articles.
- Additional control is needed over the configurability of the search index/retriever/prompt template/agent action. Examples include:
    - More granular control over which knowledge articles are included by controlling the data stream that ingests the content before building the search inde.
    - Different chunking strategy
    - Different search type (vector search instead of hybrid search)
    - Different embedding model
    - Different retriever prefilters

A search index can map to only one data source. For multiple data sources, the different data sources must be mapped to a single DMO/UDMO, since a search index can be built for a single DMO/UDMO only. Note that "files" counts as one data source, regardless of the number of files and variety of file extensions. Here's an example of a collection of data sources for RAG grounding::

- Files
- Knowledge articles
- Cases (standard Salesforce Object)
- Defects (custom Salesforce Object, holding additional information of customer interactions about defective goods)

In this example, four separate search indexes are required, each with at least one retriever.

Design Choices for RAG-Based Solution Architectures

What's the best way to organize and distribute RAG-related components in a given solution? Should you build one prompt template (and its corresponding agent action) that contains all four retrievers? Should you build a separate prompt template for each retriever? Or perhaps build two prompt templates, each with

two retrievers? Or even two prompt templates, one with three retrievers and one with a single retriever? Let's explore the rationale and tradeoffs of making such design choices.

Approach 1: One prompt template (and corresponding agent action) with all retrievers.

This approach uses these components:

- One prompt template with one agent action, and
- Multiple retrievers, each of which has a different search index, and each of which can have its own configuration (such as number of results and return fields).

When an agent action for the Agentforce reasoning engine is called, the prompt resolution *always* invokes all retrievers and augments the prompt with the results from all retrievers.

This approach is best used when:

- It's reasonably certain all the data sources are relevant to all, if not most, of the questions
- It isn't possible to define *sufficiently clearly* (with Agentforce instructions and classification descriptions) which retriever(s) should be used for which type of question

Watch out: Grounding a prompt with many retrievers can bloat the size of the prompt resolution, making it so large that it exceeds the LLM's context window. Factors include the number of results per retriever, the size of the chunks, and the number of return fields configured in each retriever. If the prompt resolution exceeds the LLM's context window, the LLM request fails with the error message "Something went wrong during generation of the response." One remedy is to reduce the output of a retriever by reducing the number of returned results or the number of returned fields. Another option is to use an LLM with a larger context window, such as GPT4 Turbo with its context window of 128,000 tokens, which is about the size of an average-length novel. Note, however, that larger prompts consume more Einstein Requests (see the corresponding Einstein Requests rate card).

Note: When ensemble retrievers soon become available (roadmap), the best solution for approach 1 will be to use them to handle multiple data sources. An ensemble retriever reranks the results and combines all result sets in a single ranking that prioritizes (at the top) the results most relevant to the query. At first, this is only possible in ADL and within other out-of-the-box features. Later, tools will enable admins to bundle multiple arbitrary retrievers into an ensemble retriever manually.

For existing solutions that use Approach 1, admins will be able to simply replace, in the prompt template, the invocation of multiple retrievers with an invocation of an ensemble retriever. The rest of the agent solution design remains intact without changes.

Approach 2: A separate prompt template for each retriever

This approach uses these components:

- One prompt template with one agent action for each data source
- One retriever for each prompt template with an associated different search index and configuration per data source (such as number of results and returned fields)

This approach requires the Agentforce reasoning engine to call multiple corresponding agent actions.

This approach is best used when:

- It's possible to specify *sufficiently clearly* (with Agentforce instructions and classification descriptions) which data source (and, thus, retriever, prompt template, and action) should be used for which type of question.
- It's possible for such a single action to generate a complete response that answers the user's question. The results retrieved from the corresponding single data source should be sufficient for the LLM to formulate this answer.

For example, suppose a data source named "Defects" contains knowledge about product defects. The corresponding instructions or classification description for this resource could be something like: "Always use the action 'Answer with known defects' when the customer asks a question about defective goods." This guides the Agentforce reasoning engine to know when to call this action at run time. Be sure to provide similarly explicit, precise descriptions and instructions for all other actions.

Watch out: **Don't** use individual actions with individual prompt templates as an alternative to approach 1. **Don't** use instructions that direct the Agentforce reasoning engine to execute a chain of multiple actions, such as: "Always use four actions to answer the user question. First, call action one for files. Second, call action two for knowledge articles," and so on. There are multiple reasons why not to use this approach:

1. Even when clearly instructed to call all actions, it's possible the Agentforce reasoning engine can consider the user intent to be fulfilled (and the answer to be given) *before* all actions are called. The result is that, at run time, the RAG solution can conceivably and unpredictably skip actions that would otherwise provide relevant information for grounding.
2. LLM instructions that define how to specifically interpret the content and generate the response are best delivered at the prompt template level. After all, instructions and classification descriptions at the agent level should primarily be used to let the Agentforce reasoning engine execute the right actions and how to leverage their outputs. They should not overextend to describe *how* these actions should be executed. With RAG, there is simply more behavioral control of the retrieval and response generation at the prompt level than at the agentic level (which is more nondeterministic).

Combining Approaches 1 and 2

If applicable to your use case, it's feasible to use both approaches in the same solution. For example, a solution can have an individual action specifically for "Defects" (approach 2), and combine the other three grounding sources ("files," "knowledge articles," and "cases") in one prompt template (approach 1).

Topic and action descriptions for RAG

Topics and actions in RAG-powered agents need descriptions, instructions, and scope delineations. All general best practices apply, as described on this help page for topics and this help page for actions. Write the instructions and scope descriptions for RAG topics and actions such that they are only selected and called-for questions within the scope of what the search index can answer.

Once the RAG action is called, the retriever from the prompt template will always fetch results from the search index and the LLM will generate a response. When no relevant content could be found in the search index, proper instructions in the prompt template will encourage the LLM to avoid hallucination (see Section 7). However, it is better to avoid that the RAG action is called for out-of-scope questions in the first place. This reduces the risk of hallucinations even further and reduces the cost and latency of the agent.

4. Hybrid Search

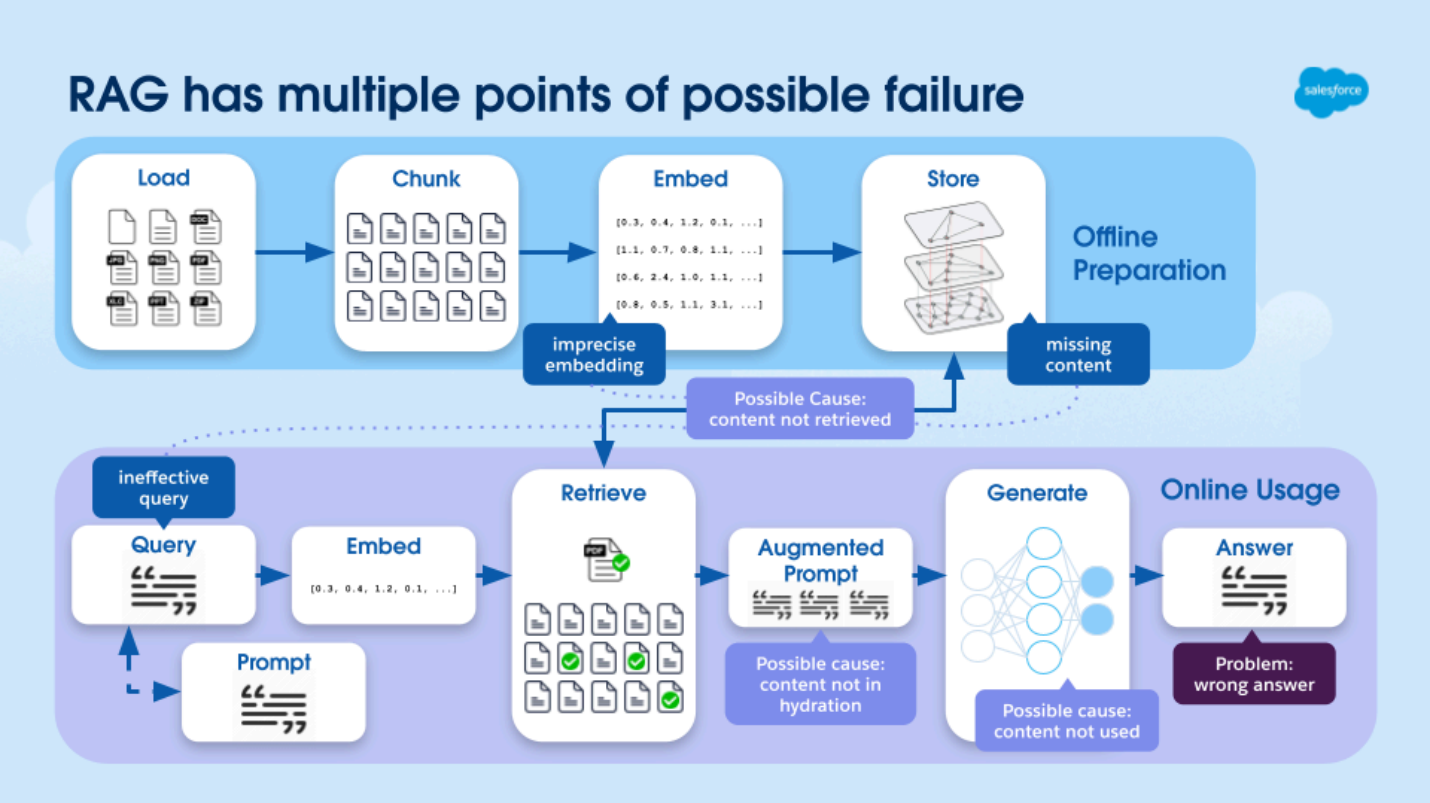
4.1 What is hybrid search?

In the RAG solution, a search index can be set up (using the search index builder) to support hybrid search. Hybrid search combines the strengths of vector search and keyword search into one search call. Think of it as two different retrieval operations from a single data source, the results of which are merged and reranked. It's analogous to ensemble retrieval (Section 3), except that hybrid search derives results from a single data source while ensemble retrievers derive results from multiple data sources.

Hybrid search combines and ranks the results of vector and keyword searches so the highest-ranked chunks are those that are both *semantically* and *lexically* similar.

By itself, vector search is strong in semantic similarity, but it can fail to recognize keywords when they matter. For example, vector search understands that "How to log in to my account?" and "How can I sign on?" are similar queries. But vector search can fail to understand that looking for "LaserPrinter TX 400" and "LaserPrinter TX 440" are also similar. Unlike keyword search, vector search doesn't match numbers well, nor does it match specific domain terms (such as a laser printer) well.

However, in combination, vector and keyword searches now reinforce each other and return the best results for questions like "What should I do if my laser printer TX 400 has a paper jam?"



Source: [Hybrid search page on help.salesforce.com]

4.2 When to use hybrid search and when not to?

Use hybrid search to retrieve context using *both* semantic and keyword similarity. Hybrid search is recommended when, for example, keywords such as product names, brands, specific terminology, or jargon are key to retrieval quality. When the user question and all retrievable content are natural, everyday language with no specific terminology, semantics or keywords, the added value of hybrid search is smaller and it is possible to rely on vector search only.

However, don't use hybrid search as a keyword search engine for categories.

For RAG solutions, vector search can be used in isolation, but keyword search cannot. Keyword search can *reinforce* the vector search results, but cannot be used standalone for lexicographic search. For search indexes, it's inadvisable to select index fields that contain only categories (for example, picklists in Salesforce).

Categories result in extremely short chunks (one word, a few words). Although these micro-chunks are also matched for semantic similarity against the user query, semantic search with single-word chunks doesn't work well because these chunks lack semantic context. As a result, the vector search part of the hybrid search becomes erratic, with inaccurate final rankings. Instead, categories are better suited as prepend fields as discussed in Section 5.1.

4.3 What are the implications of hybrid search?

Hybrid search improves retrieval results with the tradeoff of increased run-time latency and Data Cloud credit consumption.

A hybrid search operation processes queries on both vector and keyword indexes and reranks the results, consuming roughly twice as many Data Cloud services credits.

4.4. How can hybrid search be optimized?

During reranking, hybrid search combines the vector score and keyword score to produce a hybrid score upon which the final ranking is based.

Two additional ranking factors in the search index builder – popularity and recency – can influence the final rankings. In the search index builder, the user can select two fields on the (related) DMO that define these document characteristics. The final ranking takes these designations into account and ranks more popular and more recent content higher.

See this article on help.salesforce.com for more information and an example.

5. Optimize Search Indexes: Field Selection, Chunking, and Vectorization

When setting up a search index, Data Cloud performs chunking on the data before vectorizing it. Chunking decomposes the information in smaller pieces. Every chunk (and vector) represents a meaningful factoid or set of factoids. It's infeasible to represent an entire lengthy document by a single vector because this single vector can't be the semantic representation of all the content from the document.

5.1 What kinds of fields are important to RAG solutions?

Consider these four roles that fields can play in a RAG solution:

- **Index fields** (DMO indexes only). The textual contents of index fields are chunked and vectorized and used during semantic search. Note: In UDMO indexes, there is no index field selection. All the raw content from the selected file types is indexed.
- **Prepend fields** (DMO indexes only). The values of these fields are prepended to every chunk (for example, "Title" or "Product Name"). Prepending makes these field values part of the chunk, which makes them visible in prompt augmentation or in the Data Cloud Query Editor. Prepending is a

powerful mechanism to enrich chunks and improve retrieval accuracy by ensuring that key identifying fields are part of every chunk.

- **Filter fields** (UDMO/DMO indexes). Filter fields are added to the index as part of the schema of the Index DMO (that contains the vectors) but they're not vectorized. They aren't used for evaluation of semantic similarity during the search. Instead, they're used to *filter* the semantic search. Filters are defined in the retriever and can be dynamic (see Section 6.2).
- **Return fields** (UDMO/DMO indexes). Return fields can be indexed or non-indexed fields. These fields don't need to be selected when creating the search index. Returned fields are specified when building the retriever. They can come from the DMO that stores the vector or from a related DMO.

5.2. How many index fields to select, and which ones? What happens to them?

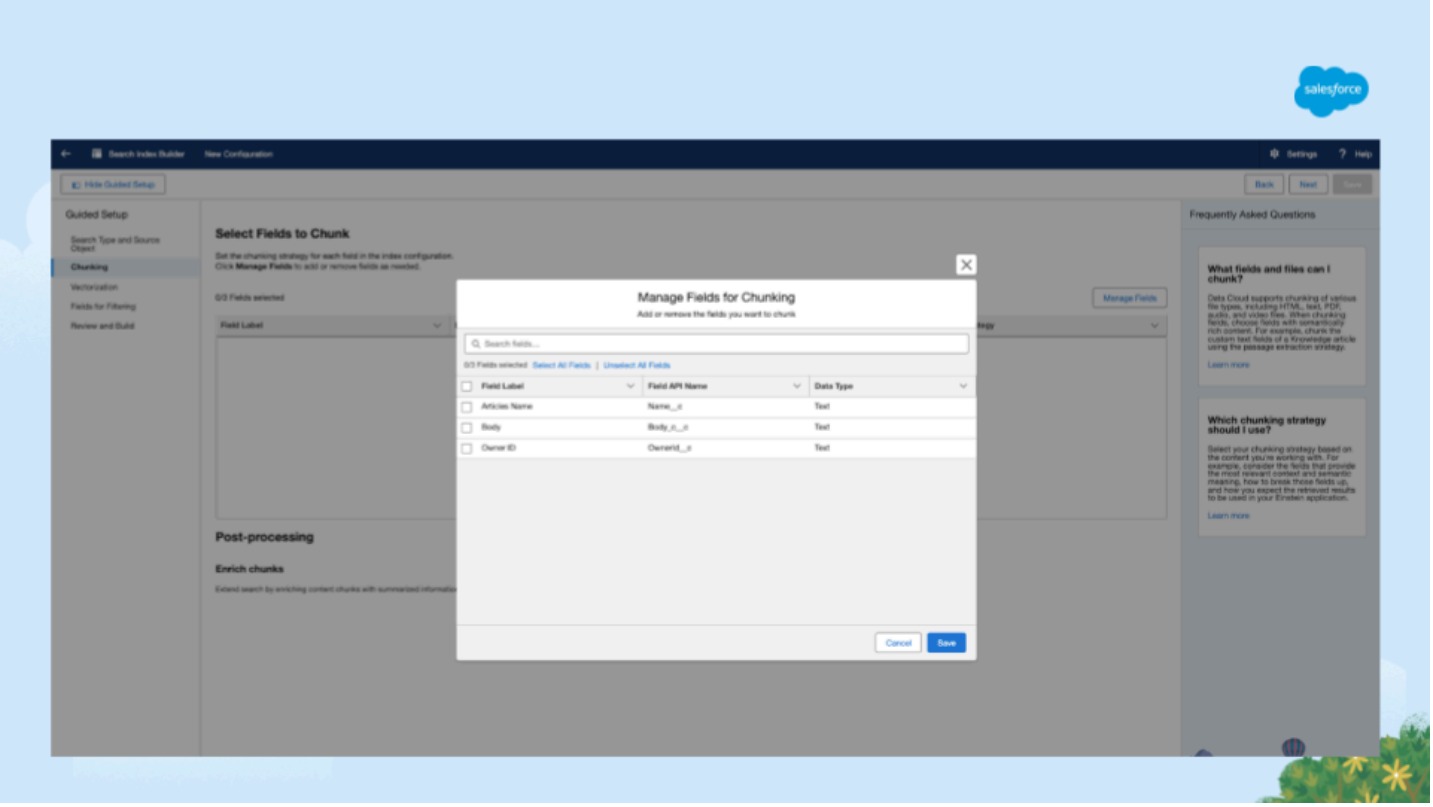
When building a search index, select index fields during step 2, chunking. Click the "Manage Fields" button. Upon search index creation, index fields are chunked, vectorized, and then used during the search process for evaluating semantic similarity to the query.

Only text fields can be selected for index fields. Select only text fields with longer, free-text contents, as opposed to categories/categorical data. Don't select categorial fields. You can select multiple fields for indexing. For example, if "Description," "Summary," "Content," and "Resolution" are selected, all corresponding vectors are stored together in the same search index. It's possible to separate vectors on the basis of the field named `DataSource__c` on the DMO of the vector. `DataSource__c` contains the original field name. Because this field is in the Index DMO, it's possible to use it in a retriever's prefilter. For example, the retriever could evaluate queries on semantic similarity to a specific field only (such as "Description" and not the "Resolution").

Caution: Don't select categorical columns as index fields. Categorial data are single-word or two-word descriptors that map to a picklist in Salesforce. To yield good results, semantic search requires a longer textual scope and more context. Recall that hybrid search *complements* semantic search with keyword search. The search index is not a keyword search engine. Instead of indexing categorical fields alone, they should be prepended to text columns that contain longer, free text contents.

Caution: Avoid selecting too many similar fields. Less is more here. Be careful not to select all text fields and avoid selecting possibly redundant fields (for example, "Summary," "Title," and "Description"). Doing so can lead to decreased recall when the search index is used without prefilters on `DataSource__c`. Because these fields all likely contain the same or very similar information, for a given query, at least three chunks from the same document can appear high in the ranking (one for each field). These bring the same information to the LLM, and when the retriever is set to retrieve, for example, nine results, only three documents will be represented in the result list. This reduces variation and may lead to documents being missed.

It is recommended that, when two or more fields represent the same content but in a different form, select the field with the least-condensed form, such as “Description” in the previous example. Consider prepending that field (see Section 5.3) with a shorter, more condensed version (such as “Title” in the previous example).



5.3 How to optimize the chunking strategy?

In search engine builder, when selecting the fields to index (DMO case) or the file types to include in the index (UDMO case), users can configure the chunking strategy, as described in [this topic in help.salesforce.com](help.salesforce.com).

Field prepending

Use field prepending to add context to chunks and make them easier to identify. For example, suppose you have a chunk that contains a sequence of troubleshooting steps. By prepending that chunk with the text “How to fix Device 123 when it shows behaviour xyz,” you make it easier to identify that content as relevant to a user’s question.

Note: Field prepending is available in DMO-based indexes but not in UDMO-based indexes.

When designing a RAG implementation, carefully consider how field prepending can benefit from the metadata in the environment.

Set up field prepending in the chunking strategy in search index builder. After selecting a field for indexing, open its dialog with chunking settings, and turn on the toggle for “prepend fields.”

Size

Another way to optimize chunking is to tune the chunk size for your solution in search index setup.

During search index creation, the platform begins by chunking down the content, as far as possible, using the semantic-based passage extraction markers described in help. The platform then lumps the granular chunks back together again until it reaches specified chunk size. The maximum configurable chunk size is currently 512 tokens, which represents about 400 to 500 words in Latin languages.

The optimal chunk size varies per solution. It depends in part on the optimization strategy that best fits the goals for a given solution.

Optimize for retrieval

Consider the information density and organizational structure of the content when optimizing the chunk sizes for retrieval. Remember that one chunk results in one vector. The entire chunk content is represented in this one vector. How many words are needed to adequately understand the meaning of a chunk? Are 400 to 500 words needed, or can fewer words sufficiently capture a self-contained, identifiable factoid of information (possibly enhanced with field prepending or chunk enrichment)?

Optimize for augmentation

Consider chunking from an augmentation perspective. What does the LLM need to generate a sufficiently usable response? Is a small, individual factoid good enough, or is more context required?

- For UDMO-based search indexes, augmentation of content typically relies on the chunk size, in which case the chunk needs to be larger to include the extra context.
- For DMO-based indexes, there are more options because additional fields can be used for augmentation. It’s even possible to augment the prompt using the original document (for example, a knowledge article) entirely instead of just the chunk. Doing so increases the prompt resolution, so consider the context window of the LLM in relation to the selected number of results. Also, such prompts increase the cost of response generation (increased the prompt size and response size consume more Einstein Requests).

5.4 What is chunk enrichment and when should it be used?

Roadmap

Chunk enrichment involves chunk types. Two new chunk types result in additional vectors.

Type	Description

PLAIN	Raw content chunks direct from the original document.
QUESTION	Contain a set of LLM-generated questions based on the plain chunk. The associated plain chunk provides the answers to these questions. All generated questions are concatenated into a single chunk before vectorization. This minimizes the possible semantic mismatch between the user intent from the conversation (phrased as a question) and the context stored in the plain chunks (phrased as answers). Question chunks improve retrieval recall and precision, especially in Q&A-related agent scenarios. Although the vectors belonging to the question chunks are retrieved, the prompt augmentation automatically occurs using the corresponding plain chunks. Therefore, the questions themselves are never augmented to the prompt.
METADATA	Contain a set of LLM-generated metadata based on the plain chunk. These are the metadata that is generated during the indexing process: ● Keywords (up to 10) ● Entities (key entities that occur in the chunk content) ● Topics (up to five main topics) ● Sentiment (positive/negative/neutral/specifically specified in the chunk) ● Title (concise and informative generated title) ● Summary (brief summary, typically between 100–250 words)

Chunk enrichment greatly improves retrieval accuracy, especially in cases where field prepending isn't possible (UDMO path) and for Q&A agent actions. Chunk enrichment provides an alternative to intensive content curation because the LLM-generated content helps improve the identification of the right chunks. The tradeoff is that chunk enrichment increases cost and latency because the retrieval operation includes a greater number of chunks.

5.5 Which embedding models should be used?

The platform supports three embedding models.

E5-Large V2 (open source)

This embedding model is recommended unless one of the other embedding models is better suited for a given solution.

Multilingual E5-Large (open source)

Use this embedding model if the content is in a language other than English. This model even preserves semantic similarity across languages; a query in French can retrieve relevant articles written in German. This embedding model supports 100 languages. The following table shows all of its supported languages,

and the number of tokens per language the model was trained with. We recommend being cautious about languages trained with less than 500 million tokens, because these languages require a thorough evaluation of the quality of the results.
Source: [Unsupervised Cross-Lingual Representation Learning at Scale]

ISO code	Language	Tokens (M)	Size (GiB)	ISO code	Language	Tokens (M)	Size (GiB)
af	Afrikaans	242	1.3	lo	Lao	17	0.6
am	Amharic	68	0.8	lt	Lithuanian	1835	13.7
ar	Arabic	2869	28.0	lv	Latvian	1198	8.8
as	Assamese	5	0.1	mg	Malagasy	25	0.2
az	Azerbaijani	783	6.5	mk	Macedonian	449	4.8
be	Belarusian	362	4.3	ml	Malayalam	313	7.6
bg	Bulgarian	5487	57.5	mn	Mongolian	248	3.0
bn	Bengali	525	8.4	mr	Marathi	175	2.8
-	Bengali Romanized	77	0.5	ms	Malay	1318	8.5
br	Breton	16	0.1	my	Burmese	15	0.4
bs	Bosnian	14	0.1	my	Burmese	56	1.6
ca	Catalan	1752	10.1	ne	Nepali	237	3.8
cs	Czech	2498	16.3	nl	Dutch	5025	29.3
cy	Welsh	141	0.8	no	Norwegian	8494	49.0
da	Danish	7823	45.6	om	Oromo	8	0.1
de	German	10297	66.6	or	Oriya	36	0.6
el	Greek	4285	46.9	pa	Punjabi	68	0.8
en	English	55608	300.8	pl	Polish	6490	44.6
eo	Esperanto	157	0.9	ps	Pashto	96	0.7
es	Spanish	9374	53.3	pt	Portuguese	8405	49.1
et	Estonian	843	6.1	ro	Romanian	10354	61.4
eu	Basque	270	2.0	ru	Russian	23408	278.0
fa	Persian	13259	111.6	sa	Sanskrit	17	0.3
fi	Finnish	6730	54.3	sd	Sindhi	50	0.4
fr	French	9780	56.8	si	Sinhala	243	3.6
fy	Western Frisian	29	0.2	sk	Slovak	3525	23.2
ga	Irish	86	0.5	sl	Slovenian	1669	10.3
gd	Scottish Gaelic	21	0.1	so	Somali	62	0.4
gl	Galician	495	2.9	sq	Albanian	918	5.4
gu	Gujarati	140	1.9	sr	Serbian	843	9.1
ha	Hausa	56	0.3	su	Sundanese	10	0.1
he	Hebrew	3399	31.6	sv	Swedish	77.8	12.1
hi	Hindi	1715	20.2	sw	Swahili	275	1.6
-	Hindi Romanized	88	0.5	ta	Tamil	595	12.2
hr	Croatian	3297	20.5	-	Tamil Romanized	36	0.3
hu	Hungarian	7807	58.4	te	Telugu	249	4.7
hy	Armenian	421	5.5	-	Telugu Romanized	39	0.3
id	Indonesian	22704	148.3	th	Thai	1834	71.7
is	Icelandic	505	3.2	tl	Filipino	556	3.1
it	Italian	4983	30.2	tr	Turkish	2736	20.9
ja	Japanese	530	69.3	ug	Uyghur	27	0.4
jv	Javanese	24	0.2	uk	Ukrainian	6.5	84.6
ka	Georgian	469	9.1	ur	Urdu	730	5.7
kk	Kazakh	476	6.4	-	Urdu Romanized	85	0.5
km	Khmer	36	1.5	uz	Uzbek	91	0.7
kn	Kannada	169	3.3	vi	Vietnamese	24757	137.3
ko	Korean	5644	54.2	xh	Xhosa	13	0.1
ku	Kurdish (Kurmanji)	66	0.4	yi	Yiddish	34	0.3
ky	Kyrgyz	94	1.2	zh	Chinese (Simplified)	259	46.9
la	Latin	390	2.5	zh	Chinese (Traditional)	176	16.6

(Roadmap) Open AI Ada 002

This embedding model is used by default when enriched chunking is turned on. It's not possible to perform enriched chunking in combination with the E5 models. The model can be used when enriched chunking isn't enabled. Ada 002 is also multilingual. However, as of this writing, OpenAI hasn't released a definitive list of supported languages. Additional testing and monitoring is recommended for uncommon languages.

5.5 RAG on documents belonging to one record

RAG often needs to be performed within the context of a given record. Examples are to search within the tasks of a particular case, or to search within the contracts of a particular account. This is possible by uploading the documents to the Salesforce record as related files. Taking the account example, the following steps are needed to set up such a (no-code) solution:

1. If not already done, set up data streams for ContentDocument, ContentDocumentLink and ContentDocumentVersion. Make sure to map the corresponding DLO to the SSOT DMOs. Make sure to create a relation on ContentDocumentLink to the source DMO (SSOT Account in this case).
2. Create a search index for Account DMO, indexing Description or any other relevant long-form text field.

3. Select “Include Attachments” on the chunking mage, and all related files from the account are also indexed. They end up in one index with the account field chunks add a prefilter to the search index (e.g., account ID).
4. Create a custom retriever with a dynamic prefilter , i.e., Account_ID = $placeholder (see Section 6.2).
5. Create flex RAG prompt template with account as input and in Prompt Builder, map the $placeholder to the account id from the prompt input (see Section 6.2 and Section 7).
6. Create an agent action around it that passes the account and the user question on to the flex prompt template

6. Configuration of the retriever

The retriever is the bridge between the search index and the prompt that it augments with context. When configuring ADL, retrievers are created automatically. To have more control over the retrieval and augmentation process, retrievers can also be created and customized manually in Einstein Studio, including for search indexes that were created using ADL.

6.1 How can prompts be grounded with data outside of the retriever/index?

The retriever can return additional fields to the chunk it retrieved. These can come from the chunk DMO or the original DMO. If the search index is created against a UDMO (unstructured data such as files), then there typically isn’t much related metadata available. This can be resolved by uploading the unstructured files to a Salesforce record as related files. Using the ContentDocument connectors, these can be brought into the search index as attachments. The search index will then contain chunks that originate from these attachments and from the selected index fields. A retriever can be configured for this search index that returns any field from the source DMO.

6.2 How can I apply filters to the results? And what are dynamic prefilters?

For custom retrievers, pre-retrieval filters (or just prefilters) can be configured to enforce specific conditions on all retrieved results, such as being written in a certain language, or belonging to a certain category. Prefiltering guarantees that the requested number of results is returned and that all results adhere to the filter condition. Filters are defined in the setup experience for retrievers and are based on fields defined in the search index builder at the time of search index creation. These fields become part of the schema of the index DMO (containing the vectors).

Note: It’s currently not possible to add prefilter fields to an existing search index.

Pre-retrieval and Post-retrieval Filters

Prefilters limit the size of the result set and they help focus the results on relevance by excluding extraneous content relative to the query. When the retriever is configured to return 10 results, it returns up to 10 results found in the search index. Results contain content only *for which the filter conditions evaluated to true.*

In contrast, post-retrieval filtering *first* retrieves the 10 results and then applies the filters. This likely reduces the size of the result set, even possibly reducing it to 0 (should none of the results the filter conditions evaluate to true). Post-filters aren't currently supported by retrievers. However, they can be formulated in pro-code solutions using Apex (see Section 9). An advantage of using post-retrieval filters is they can use *any* accessible, related field, whereas prefilters require fields that have been added to the search index for filtering purposes.

Dynamic Prefilters

In dynamic prefilters, the values of filter conditions are provided at run time. The filter condition is specified for the retriever at design time using a placeholder syntax for the value to be set upon prompt resolution. For example a filter can be `Account = $placeholder`. It's then up to the prompt engineer in Prompt Builder to map `$placeholder` to the right value from a prompt template input. For example, in a field completion template for an account field, or in a flex template that has account as input, the prompt engineer can map that placeholder to the account name or id, or whatever field has been added as identification prefilter to the search index. That way, the retriever returns only results that are tagged with that specific account.
(demo link, for now Salesforce internal only)

6.3 What is advanced retrieval mode and when should I use it?

Roadmap.
Advanced retrieval mode is a retriever feature that combines iterative retrieval with query rewriting. Using advanced retrieval mode optimizes retrieval quality, especially when user queries aren't well-formed or when a user isn't sure about what to ask for or what the search index can answer. Specifically, it consists of the following steps:

1. Run first retrieval using original user query.
2. Summarize the results of step 1.
3. LLM-based query rewriting, which uses a prompt that takes, as input, the original user query and the summarized results of step 2.
4. Run second retrieval using the rewritten query (from step 3).

RAG executes in the usual way: Augment the prompt with the results of step 4 and submit the resolved prompt to the LLM of choice to generate the final response.

7. RAG and Prompt Engineering

7.1: What does a good RAG prompt template look like?

Instructions in prompt templates are key to successful LLM generation results.

Example of a Basic Prompt Template

Let's start with a basic prompt template:

```
Unset
please answer this question:
{!$Input:question}

using this information:
{!$EinsteinSearch:ArticleRetriever_1Cx_Q8Qa1857028.results}
```

This example has two merge fields:

- `{!$Input:question}` comes from the prompt template's free text input (available on any prompt template type).
- `{!$EinsteinSearch:ArticleRetriever_1Cx_Q8Qa1857028.results}` is the insertion of a retriever that instructs the LLM to use the retrieved context. Its instructions are minimal and limited because it doesn't specify *how* the LLM should use that context.

Overly simplistic instructions risk LLM hallucinations due to various reasons.

- The LLM can choose to use some of its internal knowledge, which it considers to be more relevant than the provided context. Such internal knowledge can be out of date or less relevant.
- Or worse, the LLM can really hallucinate an answer and not even use its internal knowledge. Why? Because it considers the instruction "answer this question" to be more important than "using this information." When the LLM isn't sure how to answer given the context provided, it will do its best to *come up* with an answer that it thinks best addresses the user's intent.
- Furthermore, the prompt provides no instructions regarding the expected output or the expected level of reasoning the LLM should exercise for the given context.

In short, it's like giving a 12-year-old a geography book and saying "please study for the exam." Some students can succeed, but many require more guidance as to *how* to study and what to do with the book.

Answer Question with Knowledge Prompt Template

The standard, out-of-the-box prompt template called "Answer Question with Knowledge" provides more detailed instructions that follow common prompt design principles. In addition to what's specified in the basic prompt template described previously, this template provides:

1) Encouragement to understand the user question.

2) Explicit instructions to base the LLM response on the provided content, with encouragement to look for *relevant* information in that content.
3) What to do if the information doesn't exist in the specified source content.
4) Instructions on how to formulate the response.

Here are the instructions in the out-of-the-box prompt template, which belongs to standard actions. It uses a *dynamic retriever*:

```
Unset
###
INSTRUCTIONS

 1. Analyze the query: Carefully read and understand the user's question or
issue from the QUESTION section.
 2. Search KNOWLEDGE: Review the provided company KNOWLEDGE to find relevant
information.
 3. Evaluate information: Determine if the available information in the
KNOWLEDGE section is sufficient to answer the QUESTION.
 4. Formulate response: To generate a reply <generated_response> to the
user, you must follow these rules
 a. Find the article-chunk(s) that is most relevant to answer the user query
and VERBATIM extract the ID of the article to set <source_id> field in the
response JSON. If you are unable to find the relevant article, set
<source_id> to NONE.
 b. Use the relevant article-chunk to generate the response that exactly
answers the user's question and set the <generated response> field.
 c. If the user request cannot be answered by the knowledge provided, set
the <source_id> to NONE and <generated_response> to "Sorry, I can't find an
answer based on the available articles."
 5. Refine and deliver: Ensure your response is polite, professional,
concise and in {language} only.
 6. Review response: Make sure that you have followed all of the above
instructions, respond in the desired output format, and strictly stick to
the provided KNOWLEDGE only to formulate your answer.

 ###
KNOWLEDGE:
{!$EinsteinSearch:sfdc_ai__DynamicRetriever.results}

 ###
QUESTION:
```

```
{!$Input:Query}
```

Users have reported good accuracy with this template. For a given scenario, adding further instructions can improve the response quality.

For example, the following prompt template has a different structure:

- It provides pre-instructions for how to interpret the context
- It provides the context and the question merged in
- It concludes with how to structure the response

Note the instructions that encourage the LLM to think deeply about the offered context and to look at the question from multiple perspectives.

Unset

```
Clearly answer the user's Query directly and logically based only on
well-reasoned deductions drawn from the Context below.
Then respond to the user's Query logically, methodologically, thoughtfully
and thoroughly from multiple perspectives, emphasizing different viewpoints
based on Context with details and careful reasoning.
Provide details with organized structure in your response. Consider
alternative perspectives or approaches that could challenge your current
line of reasoning.
If you don't know how to answer the query, or if there is not sufficient
context, please respond with 'Sorry, I couldn't find sufficient information
to answer your question.'
Evaluate the evidence or data supporting your reasoning, and identify any
gaps or inconsistencies.
Finally, ask questions to clarify the user's intent while encouraging
critical thinking and self-discovery about the user's Query.
Clearly articulate with details what are facts versus what are opinions or
beliefs.
If you don't know the answer, ask questions to clarify the user's intent.
Pay attention to the entities mentioned in the user's Query and make sure
the context contains information about those entities.

Context:
{!$EinsteinSearch:ArticleRetriever_1Cx_Q8Qa1857028.results}
```

```
Query:
{!$Input:question}

Format instructions:
Format your response with Markdown structures as follows:
Start with an overview of the topic.
List the key points in a list and emphasize any critical terms using bold.
For subsequent sections, create headings and subheadings that incorporate
the subqueries implicitly.
If there are any steps or sequential data, present them in an ordered list.
End with a conclusion.
```

8. Citations (roadmap)

8.1 What are the different options for setting up citations?

Coming soon.

9. Pro-code RAG without retrievers

Retrievers connect prompt templates with search indexes. They allow users to configure a reusable, versionable, no-code query template that specifies what to retrieve from the search index based on a given search string (the user query or question). With retrievers, users can specify:

- Number of results. The number of results configured in Prompt Builder overrides the default value configured for the retriever. The number can be lower or higher.
- Return fields (directly from the Index DMO or any related DMO)
- Prefilters

9.1 What are the limitations of (no-code) retrievers?

No-code retrievers support:

- 50 results (maximum)
- Only the following prefilters:
    - Text field operators: equal to/not equal to
    - Number field operators: equal to/greater than/less than
    - Logical AND of *all* filter conditions or logical OR of *all* filter conditions (no nesting)
- No post-retrieval filter definition

For some use cases, queries require more complex expressions to the search index. Examples include:

- Different prefilter operators, such as the “contains” operator on text fields.
- Nested pre-filter conditions, including mixing logical AND and logical OR across filter conditions
- Post-filter definitions, evaluated after the results are retrieved, that provide filtering on fields that aren’t part of the search index

9.2 How to perform RAG without retrievers?

Retrievers offer a fast and easy (no-code) approach for RAG implementations. Retrievers provide additional capabilities *on top of* search index querying, such as (roadmap) advanced retrieval mode and ensemble retrievers. Over time, evolving retriever capabilities will address some of the current limitations described earlier (roadmap). However, as with every no-code artifact on the Salesforce Platform, some use cases are solved best using pro-code options.

At run time, a retriever transforms the user configuration into a Data Cloud SQL query that is used to call the vector_search or hybrid_search function. These functions can also be called from within an Apex class using the Data Cloud Connect API. Apex users have the flexibility and ability to write the query expression directly.
Refer to this help topic on hybrid search to see examples of query expressions, including examples of pre-filter expressions that the no-code retrievers do not support. Post-filters (although not represented on that page) are supported by where-clauses in SQL expressions.

Users can ground prompts using Apex classes, providing a pro-code alternative for use cases in which no-code retrievers aren’t currently an option.

Note: When the Apex class or glow returns content that would exceed the content window, that content is automatically summarized. In that case, it won’t return the underlying record/chunk data, but a summarized version.

To understand how to work with Apex classes and prompt templates, refer to Adding Apex Merge Fields to a Flex Prompt Template in Salesforce help. In the example on that page, the method `public static List<Response>` doesn’t contain the call-out to the Data Cloud connect API. However, this page provides the structure of the Apex class.

The following example creates a connection to the connect API and the query expression. The code applies a procedural filter (outside of the query expression) for user access to the retrieved content, which is currently unavailable with the no-code retriever.

JavaScript

```
public static List<Response> searchSimilarCases(List<Request> requests) {
```

```
List<Response> responses = new List<Response>();
Response response = new Response();

String caseDescription = requests[0].RelatedEntity.Description;

ConnectApi.CdpQueryInput input = new ConnectApi.CdpQueryInput();
input.sql = 'SELECT DISTINCT v.score__c Score__c, c.ssot__Id__c Id__c,
c.ssot__Subject__c Subject__c ' +
'FROM vector_search(\'case_chunk_vector__dlm\', \'' + caseDescription + '\',
\'\', 200) v ' +
'JOIN Case_Chunks__dlm cc ON v.chunk_id__c = cc.chunkid__c ' +
'JOIN ssot__Case__dlm c ON cc.parentid__c = c.ssot__Id__c ' +
'WHERE cc.column__c != \'ssot__Subject__c\' AND c.ssot__DataSourceId__c =
\'CRM\' ' +
'LIMIT 10';

ConnectApi.CdpQueryOutput output = ConnectApi.CdpQuery.queryANSISql(input);
List<Object> data = output.data;
String scs = '';
for (Object searchRecord : data) {
Map<String, Object> myMap = (Map<String, Object>)
JSON.deserializeUntyped(JSON.serialize(searchRecord));
// check for access of case record for the current user
if (SimilarCasesSearch.getUserRecordAccess((String) myMap.get('Id__c'))) {
Map<String, String> sc = new Map<String, String>();
sc.put('Id', (String) myMap.get('Id__c'));
sc.put('Similar_Case__c', (String) myMap.get('Id__c'));
sc.put('Name', (String) myMap.get('Subject__c'));
sc.put('Score__c', String.valueOf(myMap.get('Score__c')));
scs = scs + JSON.serialize(sc);
}
}
response.Prompt = scs;
responses.add(response);
return responses;
}
```

10. Selecting the right LLM for RAG

10.1 What LLM is best for RAG? What LLM parameters matter?

Depending on your use case, certain factors favor the use of some LLMs over others.

Context Window Size

It's important to select an LLM with a sufficient context window size to accommodate the size of resolved RAG prompts. Consider that one token is approximately ¾ words. For example, 100 tokens can fit around 75 words.

- OpenAi's GPT 4o and GPT 4 Turbo models have context windows of 128,000 tokens, which is largely sufficient for RAG use cases. However, GPT 4 Turbo 32,000 has a context window of 32,000 tokens.
- OpenAI's GPT 3.5 and 3.5 turbo models have smaller context windows of 16,385 tokens, which are insufficient for large prompt augmentations.
- Claude 3 Haiku has a context window of 200,000 tokens.

Reasoning Capabilities

The stronger the model, the better it can reason over the provided context. For a given use case, carefully evaluate how challenging the reasoning task will be. Is the key information roughly present in the provided context? Commonly, the greatest complexity in RAG solutions occurs more in retrieval and augmentation than in the final LLM generation. In such cases, the generating LLM won't have a very difficult task to perform anymore, and smaller models (like GPT 3.5) can be used as long as the context window is sufficiently large for the use case.

For more complex use cases in which the generating LLM still needs to reason deeper over the content (for example, combining from multiple results, transforming the input, and drawing conclusions), stronger models, such as GPT 4 (turbo), are recommended.

11. RAG in Flow

Retrievers are most commonly used in prompt templates to ground prompts. Retrievers are also used in flows for RAG. Moreover, in flows their output can be used in other ways, such as automation scenarios in which the output is used to check for existing similar content, or classification tagging (as described in Section 13.2).

RAG solutions can be implemented inside a flow. In this approach, the flow calls a retriever to fetch the grounding results, which it passes onto the prompt template that it calls subsequently. This approach gives users more control over the entire RAG process. They can set up entire pipelines of chained prompt templates, retrievers, and transformations. Instead of using a prompt template to drive the process

(including calling a flow), the flow serves as the orchestration layer and the entry point into the process. This flow-driven approach offers more advanced post-filtering of the retrieved results, such as checking for specific user access rights or other pro-code filters.

11.1 How can I use retrievers in flow? What does flow mean for RAG?

To call a retriever in a flow, add an action element to the flow. Search for the retriever name of the retriever in the list of available actions. Flow variables (such as the search string) are available as inputs to the retriever. At run time, values can be fetched from a Salesforce record, from a screen element in a screen flow, or from any other flow variable.

Calling retrievers in a flow also supports dynamic prefilters. In the formula for a dynamic filter, map the right-hand side of the equation to a flow variable. At run time, use the dynamic filter to filter the retriever results using the context of the flow (for example, country, language, category, and so on).

The following standard flow actions can be used to set up sophisticated RAG pipelines in flow.

Flow Action	Description
Detect Language	Detects the language of a query, which can be passed as a filter value to a retriever node for dynamic filtering (by language).
Transform Query for {Case/Email/Conversation}	Each of these three nodes invoke an LLM transformation that changes a case, email, or conversation into a query that is optimized for retrieval. It improves the query that the retriever passes onto the search index. For example, the conversation-to-query action avoids querying the search index with non-relevant messages such as “How can I help you?” or “How are you today?” Similarly, the case-to-query and email-to-query extract relevant information from the text to remove greetings and other text that shouldn’t be used for search.

Retriever output is formatted as a JSON array, which is not a supported type in flow. Therefore, to use the results subsequently in the flow, a processing action needs to transform the retriever output into a flow-supported type, such as a flattened String. The processor flow node can be implemented using an Apex class, as shown in the example below.

JavaScript

```
global with sharing class RetrieverProcessor {

    @InvocableMethod
    public static List<String> GetWebProduct(List<Requests> queryResults)
       {
```

```
            List<String> resultsList = new List<String>();
            for (Requests queryResult : queryResults) {
                List<String> segments = new List<String>();
                for (ConnectApi.MlRetrieverQueryResultDocumentRepresentation
document: queryResult.queryResult.searchResults) {
                for
(ConnectApi.MlRetrieverQueryResultDocumentContentRepresentation content:
document.result) {
                    if (content.fieldName.equals('Chunk')) {
                        segments.add(content.value.toString());
                    }
                }}
                if (segments.size() == 0) {
                    resultsList.add('No results');
                } else {
                    resultsList.add(String.join(segments, ','));
                }
            }
            return resultsList;
        }
    global class Requests {
        @InvocableVariable
        global ConnectApi.MlRetrieverQueryResultRepresentation queryResult;

    }
}
```

In this example, the GetWebProduct method loops through the elements of the retriever output and appends the contents of a returned field named chunk to a list of strings. The flow can then iterate through this list downstream, or pass it on to a prompt template node as input for grounding.

12. Debugging and Troubleshooting RAG-Powered agents

When an agent uses RAG to respond to a question and the response is unsatisfactory, there are various RAG-related factors to consider. To a user, the agent simply answered incorrectly, insufficiently, or perhaps not at all. Troubleshooting a poorly performing, RAG-enhanced agent involves investigating and ruling out these various points of failure one by one.

These points of failure can result from:

- Solution component or integration issues in which the agent/RAG chain is malfunctioning somewhere, or
- Qualitative issues in which all solution assets are correctly chained and called, but the level of quality in the response falls short of expectations

This section provides

- Stepwise troubleshooting recommendations (12.1) to help you determine whether the chain is intact
- Guidelines for using RAG evaluation metrics to identify more qualitative problems (12.2)

For RAG solutions with ADL, refer to this trouble shooting guide.

12.1 Troubleshooting Solution Layers

Step 1.Agent Layer: Are the agent topic and agent action being called?

Determine whether the right action within the right topic is being executed by the Agentforce reasoning engine. Use the Agentforce Agent Builder or the Testing Center to investigate and diagnose.

If the right topic isn't selected, or if the right topic is selected but the right action isn't executed, then the problem most likely occurs in the agent configuration of the instructions and classification descriptions. This is an agent problem, not a RAG issue, and therefore outside the scope of this white paper. Refer to the following instructions instead:

- help.salesforce.com on writing good Topic instructions
- help.salesforce.com on writing good Action instructions

Step 2. Agent Layer (ADL only): Is the right retriever passed to the action?

Agent Builder shows the reasoning path of the reasoning engine and its intermediate results. When using ADL and the standard action, follow the reasoning path and check whether the right retriever/grounding source is passed to the prompt template. If not, fix the agent configuration to pass the correct retriever.

Note, this approach does not apply to custom agent actions calling custom prompt templates, because the retriever call occurs entirely within the prompt template (it isn't passed in by the reasoning engine).

Step 3. Search Index Layer: Does the search index contain vectors?

There are multiple ways to determine whether the search index is correctly populated with content:

- In Data Cloud Query Editor, run a select * query against the Index DMO. Be sure to use a LIMIT 10 or similar statement.
- In Data Explorer, verify that records exist for the DMO.

- In CRM analytics (if available in the org), explore the chunks. Click the “Explore in analytics” button on the DMO home page.
- Determine whether all DMO records have been indexed, or whether the incremental update of a search index has failed. Compare the results of record count using a query similar to the following example:

```
Java
SELECT 'INDEX' AS Location, COUNT(DISTINCT rc.SourceRecordId__c) AS ArticleCount, now() AS
Timestamp
FROM <chunk DMO of the Search Index> rc
UNION
SELECT 'DMO' AS Location, COUNT(DISTINCT kav.Id__c)  AS ArticleCount, now() AS Timestamp
FROM <DMO that was indexed, e.g. Knowledge Article Version> kav
ORDER BY Location;
```

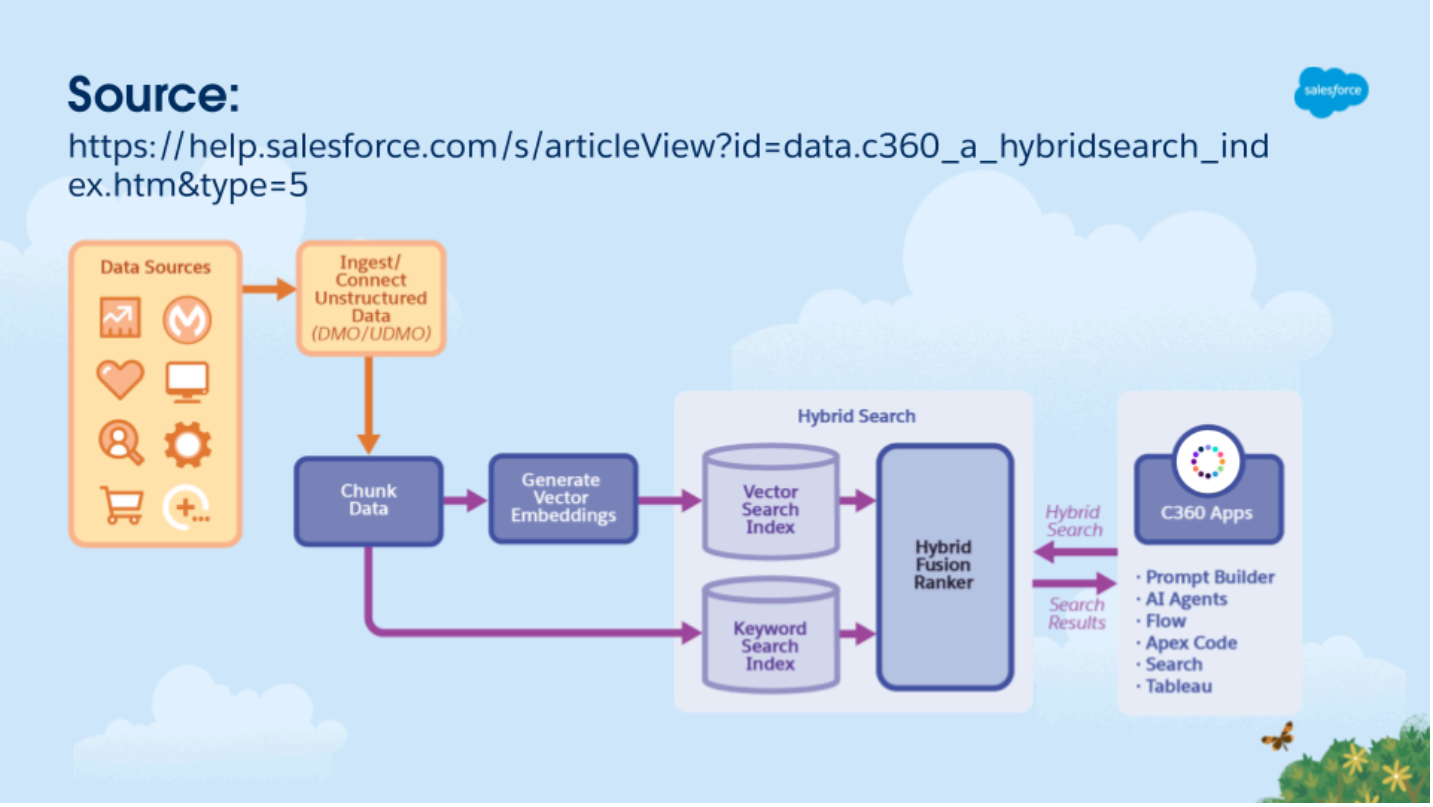
Step 4. Retriever Layer: Is the retriever augmenting the prompt with content?

In Prompt Builder, determine whether:

- the correct retriever version is added to the prompt template
- the correct result fields are activated
- the prompt resolution contains content

12.2 Interpreting RAG Evaluation Metrics roadmap

After verifying that all the assets of the RAG pipeline are connected properly, determine whether *qualitative* problems are causing answers to be incorrect, incomplete, hallucinatory, or a combination of these symptoms. Qualitative concerns can be more difficult to troubleshoot due to a myriad of possible root causes. Quality problems can arise in the retrieval, in the embedding, in the augmentation, in the response generation, and even in the original knowledge source (does the relevant content actually exist in the search index?). This figure shows where, in the RAG pipeline, quality problems can occur, and to what they can be related.



RAG evaluation quality metrics can help determine where to improve the RAG pipeline. There are three evaluation metrics calculated and shown in a dashboard. This dashboard allows drilling down to retriever level. The metrics are described below, before we dive into what they tell us when inspected jointly.

Metrics

Metric	Answers	Definition	What does it help with?
Context Relevance	How relevant is the retrieved content to the query?	LLM-based evaluation	Isolate retrieval problems
Faithfulness	How grounded is the response in the retrieved content?	LLM-based evaluation	Isolate LLM generation problems
Answer Relevance	How relevant is the answer to the query?	LLM-based evaluation	Overall response metric of the answer. Especially useful in combination with context relevance and faithfulness.

Common Patterns In Quality Metrics

High Faithfulness, Low Context Relevance

The answer is grounded in the retrieved context, but that context isn't relevant to the query. As a result, the answer relevance is also likely low. This symptom likely indicates a problem in the retrieval.
Possible remediation:

- Is the content actually present in the data?
- Is the number of results sufficiently high? Are the right result fields selected?
- Is the search string well-formed?
- Is the multilingual embedding model selected for non-English content?

Low Faithfulness, High-Context Relevance

The answer is not grounded in the context, even though that context is relevant to the query. Answer relevance is also likely low. This symptom likely indicates a problem in the LLM generation. It's possibly due to prompt engineering shortcoming, such as the LLM failing to give sufficiently strong instructions to follow the provided context.

Possible remediation:

- Is the prompt template well written? Does it follow the guidance provided in Section 7?
- Is the LLM strong enough to perform the required reasoning task? If not, consider upgrading.

High Faithfulness and High-Context Relevance, Low-Answer Relevance

The answer is grounded in the context and that context is actually relevant to the query, but the answer relevance is still low. This symptom likely indicates that there wasn't *enough* context retrieved to fully answer the query. The problem is likely in the retrieval, particularly in the recall of the retrieval.
Possible remediation:

- Is the content actually present in the data?
- Is the number of results sufficiently high? Are the right result fields selected?

13. General things worth knowing

13.1 Resolution without generating an LLM response

When setting up a RAG pipeline, admins and developers often want to resolve prompts without generating an answer by the LLM. Doing so supports analysis and optimization of the indexing/retrieval pipeline. The goal is just to observe the content retrieved by the retrievers. Generating the LLM response isn't necessary.

In Prompt Builder, add &c__debug=1 to the URL of the prompt template. This displays a toggle that lets the admin change between resolution only, response only (which provides more screen space for the response) or the standard resolution and response.

13.2 Non-RAG use cases with retrievers: document identification

Retrievers are used outside of RAG use cases. A response is not always needed. Some requirements are met by retrieving semantically similar content from the search index. Consider, for example, a case being

created in a service context. Merely showing similar cases to a Service Agent to support a case investigation can provide tremendous value without executing the entire RAG pipeline.

To set up such an automation, the recommended solution is based on flow that calls a retriever when a flow runs. As it produces the set of results similar to the query, the sources of these results (e.g., cases, articles), can be presented to the user.

13.3 Non-RAG use case with search indexes: classification

Use a search index to conduct text classification, such as intent detection, topic annotation, or case classification. Classification use cases are often solved using a training data set (inputs and their class labels). Instead of training a text classifier with this training data set, the text can be vectorized. Store these inputs in Data Cloud as records of a DMO so that they can be embedded in a search index. A search operation is then based on the semantic similarity between the query and the embedded inputs. However, instead of returning the chunks of the "training" inputs, the search returns the original class labels. When the number of results is sufficiently large (say 50 or 100), it's possible to conduct a "majority" vote and see which class labels occur most frequently within that set of results. Ordering the class labels by their frequency in the result set provides classification suggestions. Either select the most occurring class label, or select, say, the top three class labels to the user.

This scenario requires supplemental Apex code because the retriever does not support the SQL query used (based on COUNT). This code example uses a query to count the frequency of each class label occurring in the top-50 results, ordering by that count statement, and selecting the top-most class label as the outcome.

Unset

```
ConnectApi.CdpQueryInput input = new ConnectApi.CdpQueryInput();
input.sql = 'SELECT r.Label_c__c Label, COUNT(r.Label_c__c) AS counter FROM
vector_search(table(Intent_Training_index__dlm), topic,'' , 50) v JOIN
Intent_Training_chunk__dlm c ON v.RecordId__c = c.RecordId__c JOIN
Intent_Training__dlm r ON r.Id__c = c.SourceRecordId__c GROUP BY
r.Label_c__c ORDER BY counter DESC LIMIT 1;

ConnectApi.CdpQueryOutput output = ConnectApi.CdpQuery.queryANSISql(input);
```

Appendix

Forward-looking statements



Summary of provided best practices

1. Ingest structured content into DMOs, not UDMOs. Chunk and vectorize long-form text columns and use the other columns (metadata) for field prepending, prefiltering and return fields. (Section 2.1)
2. Curate content before indexation. (Section 2.2)
    a. Use a high level of detail with thorough explanations.
    b. Provide real-world examples.
    c. Structure articles with logically connected sentences and paragraphs/headers.
    d. Spread structured content over fields of sObjects, such as knowledge articles.
    e. Annotate media.
    f. Keep content focussed and aligned with user questions.
    g. Use titles, headings, and subheadings.
    h. Perform knowledge audits and apply governance.
    i. Convert complex tables to JSON or HTML and split long tables.
3. Use a multilingual embedding model for multilingual content. This maintains semantic similarity across languages. More languages are supported for RAG indexing and retrieval than for prompt

response generation. The LLM will understand all these input languages, but for response generation, trust layer and agent conversations a smaller number of languages is supported. (Section 2.3)

4. Only use ADL when it *exactly* matches the use case. ADL is a fast track to RAG-powered agents, but has limited configurability and content support. Alternatively, set up the RAG solution manually. (Section 3.1)
5. Combine multiple retrievers in a single prompt template if all retrievers are relevant to all user queries. Otherwise, split the retrievers over multiple prompt templates and their corresponding agent actions. Provide appropriate instructions for the agent actions so that the right action is called. (Section 3.2)
6. Avoid calling RAG actions for out-of-scope questions by writing proper descriptions, instructions and scope. This reduces the risk of hallucination and reduces cost and latency of the agent. (Section 3.2)
7. Use hybrid search when semantic similarity search needs to be improved with keyword search. Hybrid search strengthens semantic search with keyword similarity such as product names, company terminology, and jargon. (Section 4.2)
8. Don't use hybrid search as a standalone keyword search engine for categories. The vector search that runs in parallel distorts the results due to a lack of semantic context. (Section 4.2)
9. Consider the necessity of hybrid search. Hybrid search improves retrieval results with the tradeoff of increased run-time latency and cost. (Section 4.3)
10. Use ranking factors recency and popularity to further improve search results. (Section 4.4)
11. Only index fields that are long-form text containing at least one semantically coherent, full sentence. Use the other fields as prepending fields, prefilter fields and return fields. (Section 5.1)
12. Index as few fields as possible. When multiple fields contain similar content, select the most detailed one only. (Section 5.2)
13. Use enriched indexing for more complex content that requires the generation of annotation with metadata and questions for accurate retrieval. (Section 5.4)
14. Use the default embedding model E5-Large-V2 unless the content is multilingual. (Section 5.5)
15. Use additional return fields to improve prompt augmentation. (Section 6.1)
16. Use (dynamic) prefilters to improve retrieval precision and focus content on specific records. (Section 6.2)
17. Use advanced retrieval mode if users enter imprecise/high-level search queries. (Section 6.3)
18. Use prompt engineering techniques to improve the quality of generated responses. Instructions about following only augmented content, reasoning over that content and output formatting have a large impact on response quality. (Section 7.1)
    a. Encouragement to understand the user question.
    b. Explicit instructions to base the LLM response on the provided content, with encouragement to look for *relevant* information in that content.
    c. What to do if the information doesn't exist in the specified source content.
    d. Instructions on how to formulate the response.
19. Use prompt augmentation with Apex and/or flow (instead of no-code retrievers) when nested post-filters are required, nested prefilters or prefilters with unsupported field types and/or operators. (Section 9.1)

20. Select the right LLM for the RAG operation based on 1) context window and 2) reasoning capabilities. (Section 10.1)
21. Use flow to build more complex RAG pipelines by using actions such as retriever and prompt template invocation. The flow becomes the orchestrator of the RAG pipeline instead of the prompt template. (Section 11.1)
22. Methodically follow the chain of RAG assets when troubleshooting the RAG solution. (Section 12.1)
    a. Agent layer
    b. ADL layer
    c. Search index layer
    d. Retriever layer
23. In case of quality problems, use RAG metrics to identify the area of improvement. (Section 12.2)
    a. Context relevance for retrieval problems
    b. Faithfulness for augmentation and prompt problems
    c. Answer relevance for general RAG problems
24. Use resolution-only mode in Prompt Builder when during debugging of the RAG solution a response generation isn't required. This saves time, cost and energy. (Section 13.1)
25. Use a search index for text classification use cases by indexing previously classified examples. Classify a new text based on the class labels of its 50 or 100 most similar search results. (Section 13.3)

Acknowledgements

The author would like to thank the proofreaders for their invaluable help in shaping the content of this white paper: Eric Ivory-Chambers, Robin de Bondt, Jan van den Broeck, c Raigon, Vahe Ayvazyan, Giuseppe Cardace, Praveen Gonugunta, Kathryn Baker Parks, Debbie Symanovich.